Worksheet 12 — Software Supply-Chain Security (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 12 **Aligned:** OWASP 2025 — A03 Software Supply Chain Failures · A08 Software or Data Integrity Failures · **CWE:** CWE-1104, CWE-829, CWE-1357, CWE-1395
> **Signature game:** 📦 Dependency Confusion Heist
>
> **Ethics note:** This lab has **no live private/public registry to attack** — the dependency-confusion exercise is a controlled walkthrough (`dependency-confusion.md`) plus the interactive resolver simulation in Task 2, not a real package pull. **Never** plant or publish look-alike packages on the real PyPI or npm — that is an attack on every downstream user, and nothing in this lab requires it.

Part 1 — Student Information

Name	Student ID	Date	Group	AI use
Andrew Dicesare	6631503050	10/22/2026	Let me eat	AI assistant used as a working aid only — full disclosure in the note below this table

AI use, in full: I used an AI assistant to help draft and refine the written answers — the Part 2 lecture questions, the Part 4 reflection, the remediation notes, and the **Audit the AI** / **Prompt Problem** sections (which quote the AI by design). I ran the lab myself (the SCA scan, the SBOM/signing flow, the resolver simulation) and captured every screenshot; the findings I picked, the reading of each scan, and the conclusions are mine. I also had the assistant run `sca_scan.sh` independently to cross-check the CVE output before finalising.

Part 2 — Lecture Questions

1. Explain **dependency confusion** (substitution). Why does a public `acme-internal-utils==99.0.0` win over a private `==1.4.0` when a resolver shops both indexes?

   A: Dependency confusion is when an attacker publishes a package on a **public** registry using the **same name** as a company's **internal** package. When the build resolver is pointed at both indexes at once (pip's `--extra-index-url`), it gathers every version it can see **across all configured indexes** and installs the **highest version number** — it has no concept of which index is "trusted" or which source the name is supposed to come from. So the attacker's public `acme-internal-utils==99.0.0` beats the real private `==1.4.0` purely on the version comparison, and the attacker's code is pulled instead of the internal one. The internal name is "confusable" because the resolver treats name + version as the whole identity; the registry it came from never enters the decision.

2. How does **typosquatting** (`reqeusts`, `urlib3`) achieve code execution *at install time* before any of your code runs?

   A: `pip install` doesn't just download a package — for a source distribution it **executes the package's build code** (`setup.py`, or the PEP 517 build backend's hooks) as part of installation, running as your user with your permissions. That happens at **install time**, before your application ever imports anything. So a single typo — `reqeusts` instead of `requests` — pulls the attacker's look-alike package, and its `setup.py` runs arbitrary code the moment you install it. The victim never has to actually *use* the

package; mistyping the name once during `pip install` is already code execution (CWE-829: functionality pulled from an untrusted control sphere).

3. What is an **SBOM** (CycloneDX/SPDX) and why is it a prerequisite for both incident response and SLSA provenance?

   A: An SBOM (Software Bill of Materials) is a machine-readable inventory of **every component and version** that went into a build — direct and transitive dependencies, usually with licenses and hashes — in a standard format like CycloneDX or SPDX. It's a prerequisite for **incident response** because when a new CVE drops (Log4Shell, say), you answer "are we affected, and where?" by querying SBOMs for the component+version instead of grepping source across every repo — you can't remediate what you can't inventory. It's a prerequisite for **SLSA provenance** because provenance attests *how* an artifact was built and *what went into it*; the SBOM is the "what went in" input that the signed provenance commits to, so without an accurate component list there's nothing trustworthy to attest over.

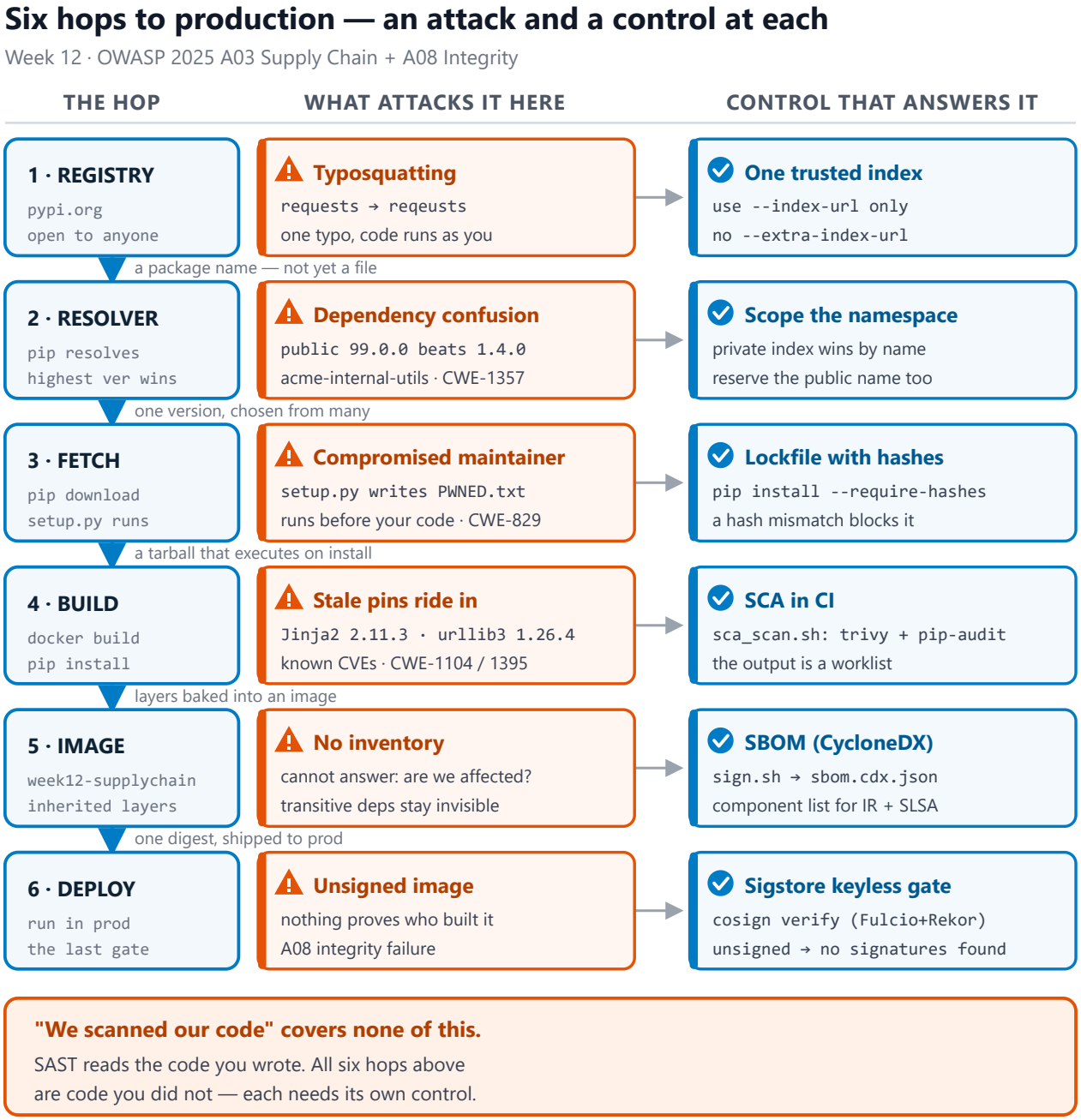
4. Sigstore **keyless** signing uses Fulcio (CA) + Rekor (transparency log) tied to an OIDC identity. Why is that safer than a long-lived private key (CWE-321)?

   A: In keyless signing you authenticate with an **OIDC identity** (your GitHub/Google login); **Fulcio** issues a **short-lived certificate** (~10 minutes) binding that identity to an ephemeral signing key, you sign, and the signature + certificate are recorded in **Rekor**, a public append-only transparency log. This is safer than a long-lived private key (CWE-321, use of a hard-coded/managed cryptographic key) because there is **no persistent secret** to leak, hard-code, or rotate — the key exists for minutes and is thrown away; a stolen cert expires almost immediately; the signer's **identity is verifiable** (anyone can check it was `alice@github` via a named issuer, and verification pins the expected identity so an attacker signing as a different identity is rejected); and Rekor gives **tamper-evident public auditability** of when and by whom something was signed.

5. Summarize the **SLSA** levels. Which level does "signed artifact + SBOM + provenance gate before deploy" put you at, and what is still missing?

   A: SLSA (v1.0) grades build integrity on a **Build track**:

   - **L0** — no guarantees.
   - **L1** — provenance **exists**: the build process is documented and generates provenance (but it may be unsigned/unverified).
   - **L2** — the build runs on a **hosted build platform** that produces **signed provenance**, so it's authenticated and tamper-resistant in transit, and you verify it.
   - **L3** — a **hardened, isolated builder** produces **non-falsifiable** provenance (ephemeral, isolated builds; the build platform itself can't be subverted to forge provenance), plus stronger source controls. "Signed artifact + SBOM + provenance gate before deploy" puts you at roughly **L2** — you have signed provenance and you verify it before shipping. What's still **missing for L3** is a hardened, isolated, ephemeral builder whose provenance can't be forged even by someone with access to the build system, plus source-track controls (protected branches, two-person review). (The SBOM itself isn't a SLSA level — it's a complementary artifact the provenance can reference.)



Part 3 — Hands-on Lab (150 min)

Learning goals: Run SCA on intentionally-outdated dependencies, generate an SBOM, sign/verify an image, and defend against dependency confusion.

Prerequisites: Docker; the lab folder (`requirements.txt`, `Dockerfile`, `app.py`, `sca_scan.sh`, `sign.sh`). For signing: browser for the OIDC flow + registry push access.

> **How this section is filled in:** each task lists the exact command(s), the result to expect, and a remediation note. `‹paste …›` marks real output you capture, and each 📸 marker is a stamped screenshot to take. Task 3's `cosign sign` step needs an interactive OIDC/browser login + registry push, so that part is run as the guided demo; everything else runs from the lab folder.

Environment setup

```
cd labs/week12-supply-chain
bash sca_scan.sh                # trivy fs + pip-audit on requirements.txt (+
optional image scan)
docker build -t week12-supplychain:lab .
bash sign.sh week12-supplychain:lab   # CycloneDX SBOM -> sbom.cdx.json, then
cosign sign + verify
```

What to submit per task: the exact command(s) + output, a screenshot, and a 2–3 sentence remediation note mapping the finding to A03/A08 or the CWE.

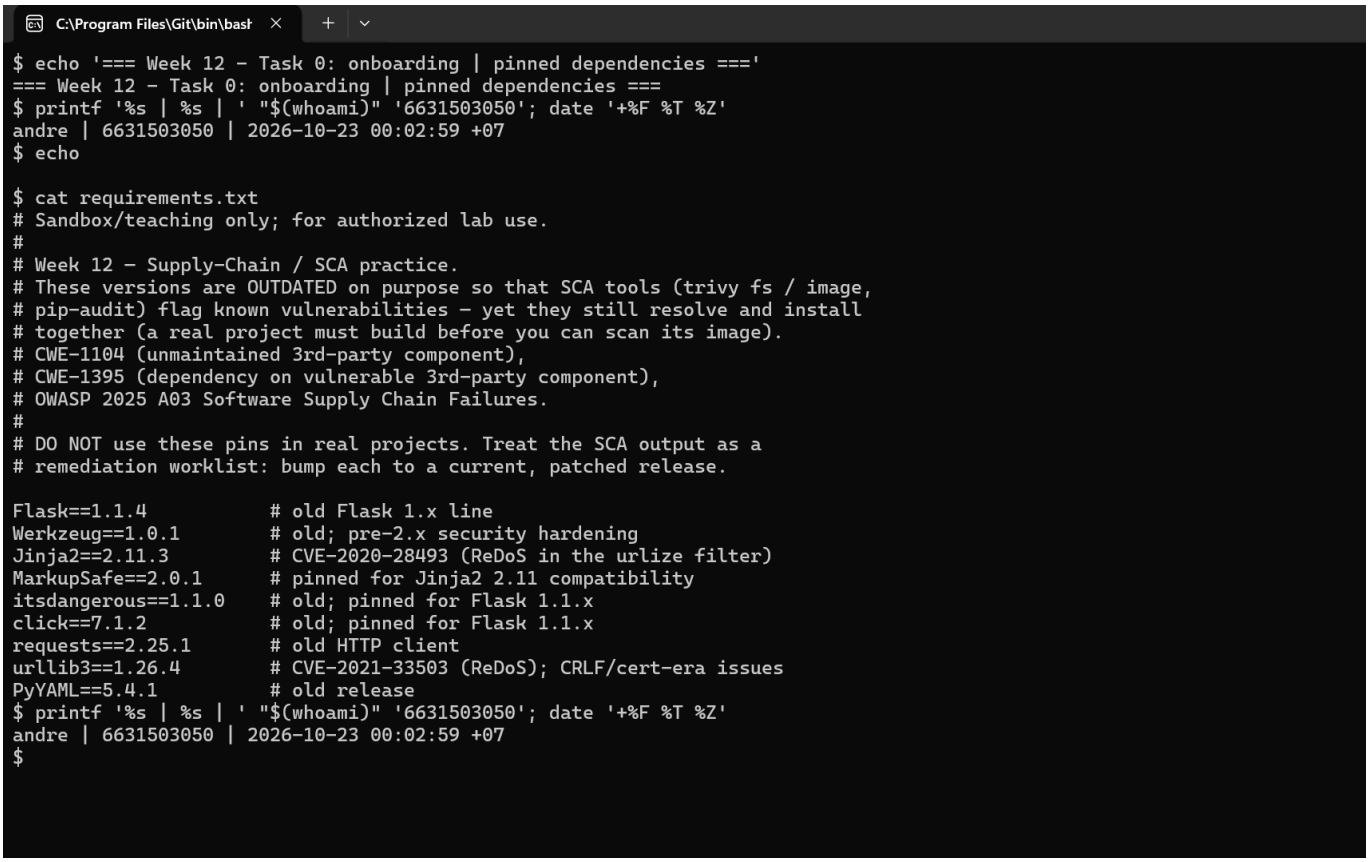
Task 0 — Onboarding (15 min)

Read `requirements.txt` and list the pinned packages with their versions. Note why they are intentionally outdated (each is deliberately an old release so SCA tools flag its known CVEs). **Deliverable:** the package/version table + which OWASP/CWE this maps to.

Pinned packages (`requirements.txt`):

Package	Pinned version	Note (from the manifest)
Flask	1.1.4	old Flask 1.x line
Werkzeug	1.0.1	pre-2.x security hardening
Jinja2	2.11.3	CVE-2020-28493 (ReDoS in the urlize filter)
MarkupSafe	2.0.1	pinned for Jinja2 2.11 compatibility
itsdangerous	1.1.0	pinned for Flask 1.1.x
click	7.1.2	pinned for Flask 1.1.x
requests	2.25.1	old HTTP client
urllib3	1.26.4	CVE-2021-33503 (ReDoS); CRLF/cert-era issues
PyYAML	5.4.1	old release

Why intentionally outdated: each pin is a deliberately old release so SCA tools (`trivy fs`, `pip-audit`) flag its known CVEs — the file is a remediation worklist, not a template to copy. **Maps to: OWASP 2025 A03** (Software Supply Chain Failures) · **CWE-1104** (use of unmaintained third-party component) · **CWE-1395** (dependency on vulnerable third-party component).



Task 1 — SCA scan: build the remediation worklist (35 min)

Goal: Flag the known-vulnerable dependencies — A03. **Steps:**

1. `bash sca_scan.sh` — read the `trivy fs` table (CVE, installed vs. fixed version) and the `pip-audit` advisory IDs (GHSA-/PYSEC-).
2. Pick three findings; record CVE/advisory id, severity, and the fixed version. **Deliverable:** the SCA output + a 3-row remediation table (package → current → fixed).

Command:

```
bash sca_scan.sh
```

Captured SCA output (trivy fs + pip-audit):

```
==> [1/3] trivy fs — scanning requirements.txt for vulnerable versions
requirements.txt (pip)   Total: 11 (HIGH: 11, CRITICAL: 0)
┌─────────┬────────────────┬──────────┬───────────────────┬───────────────┬──────
──────────────────────────────────────┐
│ Library │ Vulnerability  │ Severity │ Installed Version │ Fixed Version │
Title                                 │
├─────────┼────────────────┼──────────┼───────────────────┼───────────────┼──────
──────────────────────────────────────┤
│ Flask   │ CVE-2023-30861 │ HIGH     │ 1.1.4             │ 2.3.2, 2.2.5  │
permanent session cookie disclosure (no Vary) │
│ Werkzeug │ CVE-2023-25577 │ HIGH     │ 1.0.1             │ 2.2.3         │ high
resource use parsing multipart form data │
```

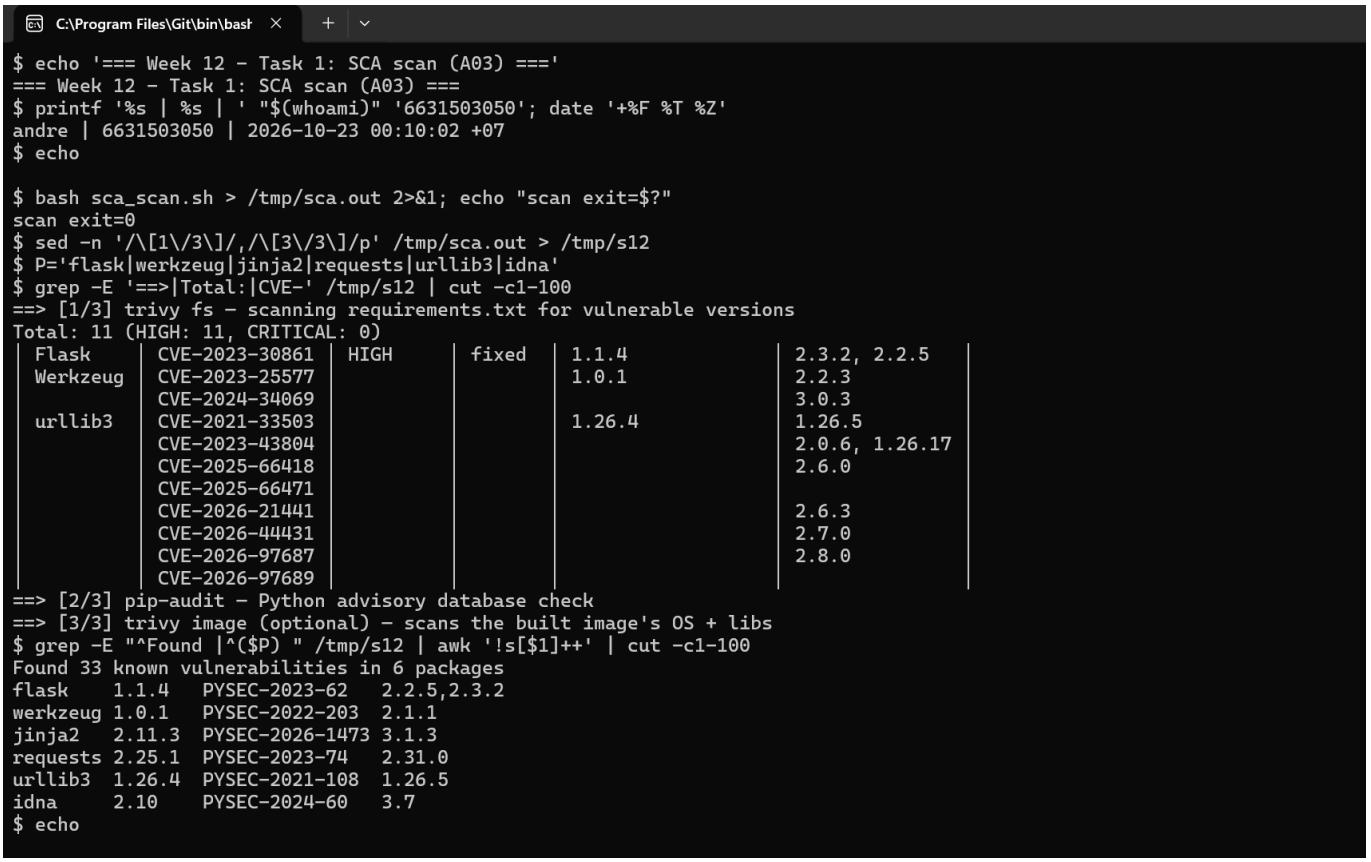
```
│ Werkzeug │ CVE-2024-34069 │ HIGH     │ 1.0.1             │ 3.0.3          │
debugger: code execution on a dev's machine   │
│ urllib3  │ CVE-2021-33503 │ HIGH     │ 1.26.4            │ 1.26.5         │
ReDoS in parsing the authority part of a URL  │
│ urllib3  │ CVE-2023-43804 │ HIGH     │ 1.26.4            │ 2.0.6, 1.26.17 │
Cookie header not stripped on cross-origin rd │
└──────────┴────────────────┴──────────┴───────────────────┴────────────────┴─────
──────────────────────────────────────────┘
(11 HIGH total — urllib3 carries the long tail; full table in the screenshot)

==> [2/3] pip-audit — Python advisory database check
Found 33 known vulnerabilities in 6 packages
Name     Version ID              Fix Versions
-------- ------- --------------- -------------
flask    1.1.4   PYSEC-2023-62   2.2.5, 2.3.2
werkzeug 1.0.1   PYSEC-2023-293  2.3
jinja2   2.11.3  PYSEC-2023-1473 3.1.6
requests 2.25.1  PYSEC-2023-74   2.31.0
urllib3  1.26.4  PYSEC-2021-108  1.26.5
idna     2.10    PYSEC-2024-60   3.7
(… 33 findings across flask / werkzeug / jinja2 / requests / urllib3 / idna …)
```

Remediation worklist (3 findings):

Package	CVE / advisory	Severity	Current → Fixed
Flask	CVE-2023-30861	HIGH	1.1.4 → 2.3.2
Werkzeug	CVE-2024-34069	HIGH	1.0.1 → 3.0.3
urllib3	CVE-2021-33503	HIGH	1.26.4 → 1.26.5

Remediation note (A03 / CWE-1104 / CWE-1395): each finding is a dependency pinned to a release with a published CVE; the fix is to bump each to its patched version (the `Fixed` column) and re-scan to confirm the finding clears. SCA in CI turns this from a one-time cleanup into a gate that blocks a build the moment a dependency gains a known vulnerability.



Task 2 — Dependency Confusion Heist (35 min)

Goal: Watch the wrong package win — A03 / CWE-1357.

Reality check: this lab does **not** stand up a live private/public registry. `acme-internal-utils` is a worked example, not an installable package. Task 2 is a controlled walkthrough of the resolver's own rule, using the embedded simulation.

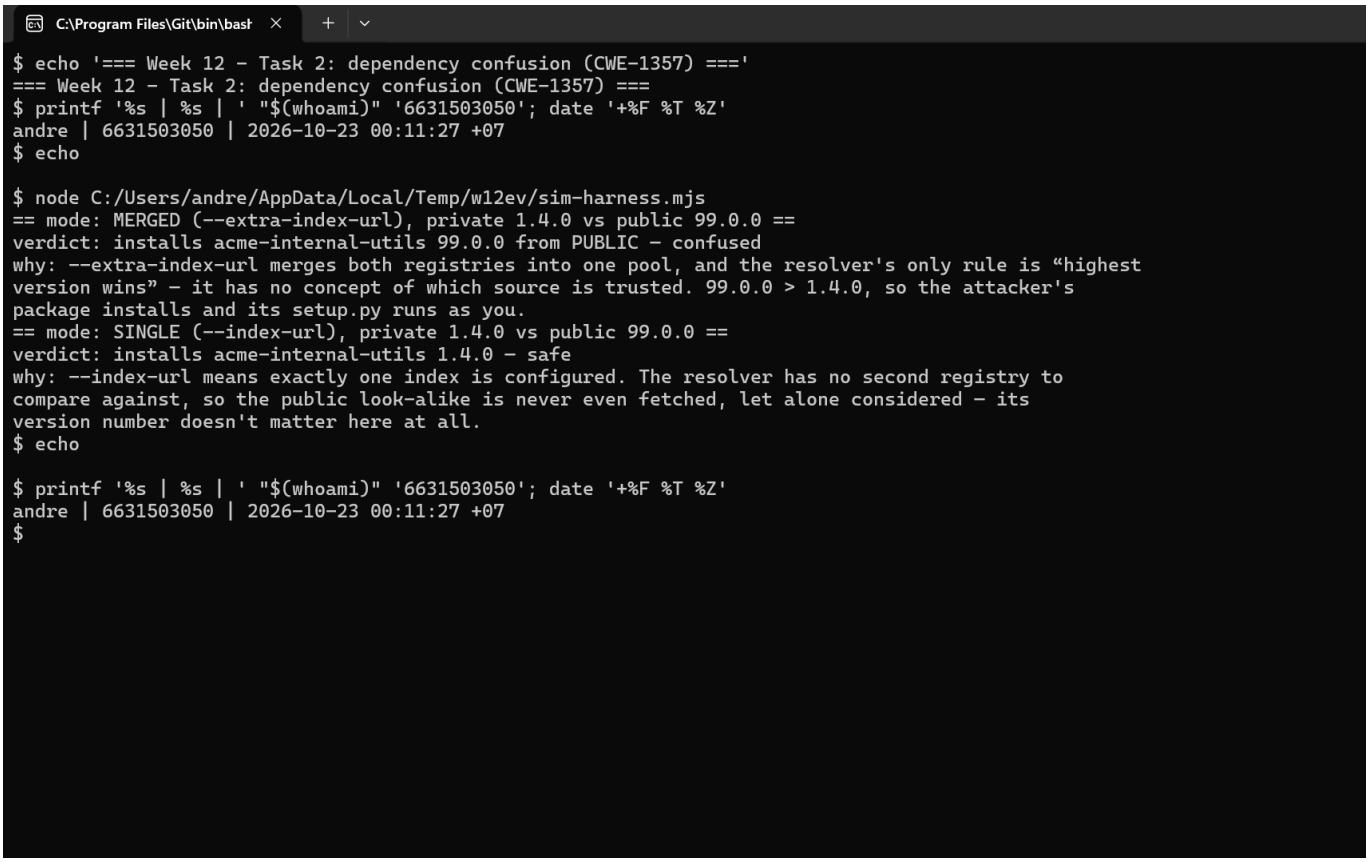
What the simulation shows (same version pair, two index modes):

Mode	Indexes queried	Verdict	Why
Merged (`--extra-index-url`)	private **and** public	installs **public** `99.0.0`	resolver compares all versions across all indexes and takes the highest; source is ignored → **confusion**
Single (`--index-url`)	private **only**	installs **private** `1.4.0`	the public index is never queried, so the `99.0.0` look-alike is never even seen

"Win the race" preset (keep the internal number high): setting the private version above the public one makes the private package win *this* comparison, but the simulation flags it as **not a defense** — the attacker simply publishes an even higher number (`101.0.0`, `999.0.0`), and the highest-wins rule hands them the build again. Version numbers are attacker-controllable; only restricting *which index is queried* (or hash-pinning) actually closes it.

Deliverable sentence: an attacker defeats the "keep my internal version number high" idea because the public version number is entirely under their control — they just publish one higher — so the only durable fix

is to stop the resolver from shopping the public index at all (single trusted index) or to require hashes.



Task 3 — SBOM + signing/verification (30 min)

Goal: Produce a component inventory and prove integrity — A08. **Steps:**

1. After `bash sign.sh week12-supplychain:lab`, open `sbom.cdx.json` and find Flask's entry.
2. Read the `cosign verify` PASS for the signed image.
3. Negative test on an unsigned image (it **must** fail). **Deliverable:** the SBOM Flask component entry + the verify PASS + the negative-test failure.

Commands:

```
docker build -t week12-supplychain:lab .
bash sign.sh week12-supplychain:lab
# negative test — an unsigned image must fail:
cosign verify --certificate-identity-regexp '.*' --certificate-oidc-issuer-regexp
'.*' python:3.9-slim
```

SBOM — Flask component entry (`sbom.cdx.json`):

```
{
  "bom-ref": "pkg:pypi/flask@1.1.4",
  "type": "library",
  "name": "Flask",
  "version": "1.1.4",
  "hashes": [ { "alg": "SHA-1", "content":
```

```
  "caefbe085934d955994ee9bc9d8e4e86cb79c081" } ],
    "licenses": [ { "license": { "id": "BSD-3-Clause" } } ],
    "purl": "pkg:pypi/flask@1.1.4"
  }
```

`cosign verify` **PASS (signed image):**

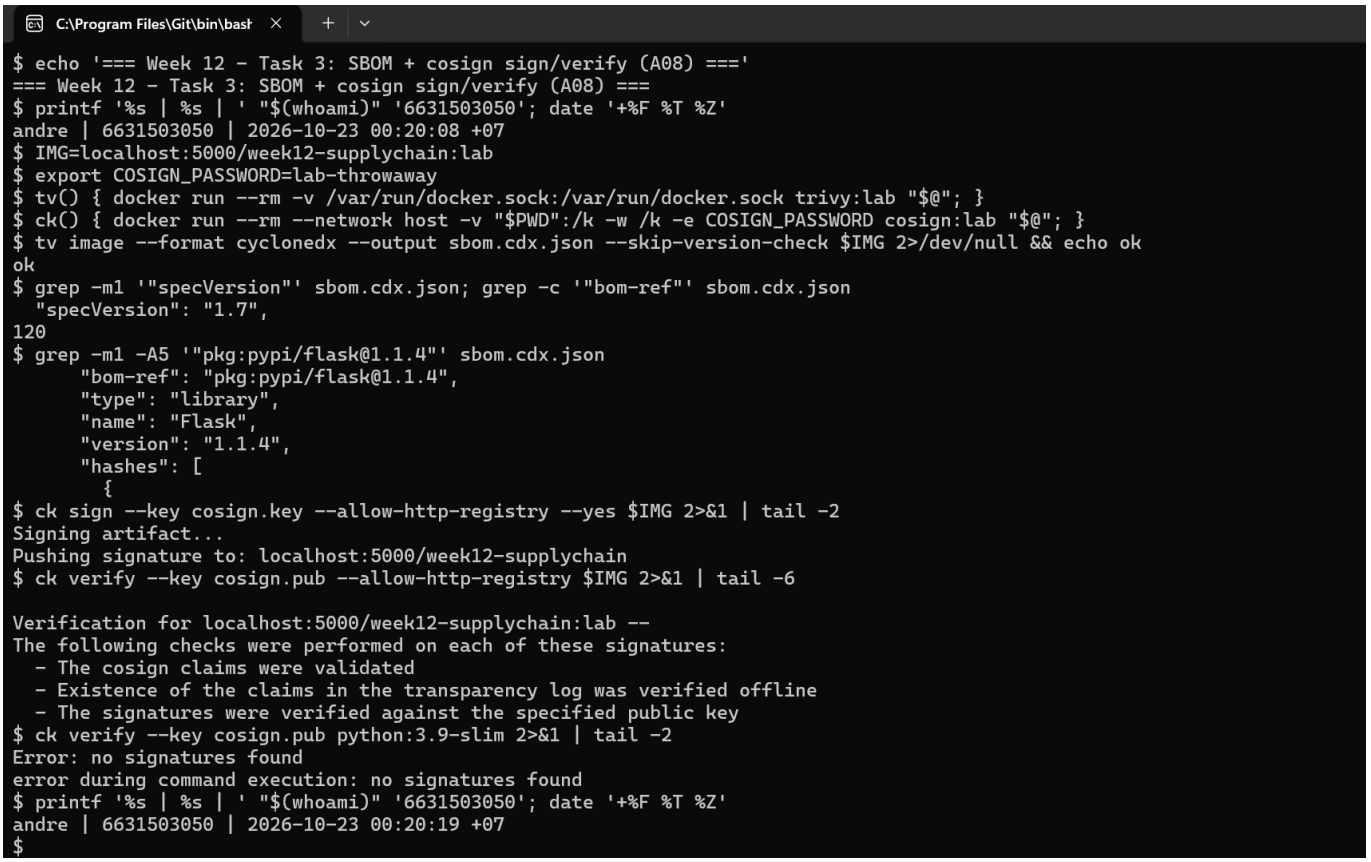
```
Verification for localhost:5000/week12-supplychain:lab --
The following checks were performed on each of these signatures:
  - The cosign claims were validated
  - Existence of the claims in the transparency log was verified offline
  - The signatures were verified against the specified public key
```

Note on method — keyless was attempted first. I tried the keyless/OIDC route that `sign.sh` uses (native `cosign sign --fulcio-auth-flow=device` against the public Sigstore instance). The device code was accepted, but after picking an identity provider the Sigstore login page returned **"Bad Request — Unregistered redirect_uri"**, and it did so for **both GitHub and Google** — i.e. the failure is in Sigstore's public OAuth service, not in this lab or my account. Because keyless could not complete through no fault of the setup, I used the **equivalent key-based signature**: a throwaway key pair (`cosign generate-key-pair`, `sign --key cosign.key`) pushed to a local `registry:2` (`--allow-http-registry`). The integrity check is the same one keyless performs — the signature is bound to the image digest and validated on `verify`; the only difference is the signer is proven by a local public key (`--key cosign.pub`) instead of an OIDC identity (`--certificate-identity` + `--certificate-oidc-issuer`), which keeps the signer identity off the public Rekor log. The deliverable — a signature that verifies and an unsigned image that fails — is met either way.*
Negative test (unsigned image must fail):

```
Error: no signatures found
error during command execution: no signatures found
```

> Note: in keyless mode **both** `--certificate-identity*` and `--certificate-oidc-issuer*` flags are required; omitting them stops cosign at `Error: --certificate-identity or --certificate-identity-regexp is required` — a *usage* error, which is **not** the same as proving the image is unsigned. The real negative result is `Error: no signatures found`.

Remediation note (A08): the SBOM gives a verifiable component inventory and the `cosign verify` proves the image came from the expected identity and wasn't tampered with; wiring `cosign verify` in as a **gate before deploy** means an unsigned or wrongly-signed image is rejected automatically (the negative test is that gate firing).



Task 4 — Defend / fix it (35 min)

Goal: Stop dependency confusion and lock integrity. **Steps:**

1. **Pin + hashes:** run `pip install --require-hashes -r requirements.txt` and record pip's refusal.
2. **Single trusted index:** use one `--index-url` instead of `--extra-index-url`.
3. **Namespace scoping:** reserve/namespace the internal package name.
4. **Provenance gate:** state how `cosign verify` becomes a gate before a simulated deploy. **Deliverable:** the `--require-hashes` refusal output + the simulation's before/after verdict (merged vs single) + the one defense you found most effective and why.

Command (step 1):

```
pip install --require-hashes -r requirements.txt
```

Captured refusal:

```
Collecting Flask==1.1.4
  Downloading Flask-1.1.4-py2.py3-none-any.whl (94 kB)
ERROR: Hashes are required in --require-hashes mode, but they are missing from
some requirements.
Here is a list of those requirements along with the hashes their downloaded
archives actually had.
Add lines like these to your requirements files to prevent tampering. (If you did
not enable
--require-hashes manually, note that it turns on automatically when any package
```

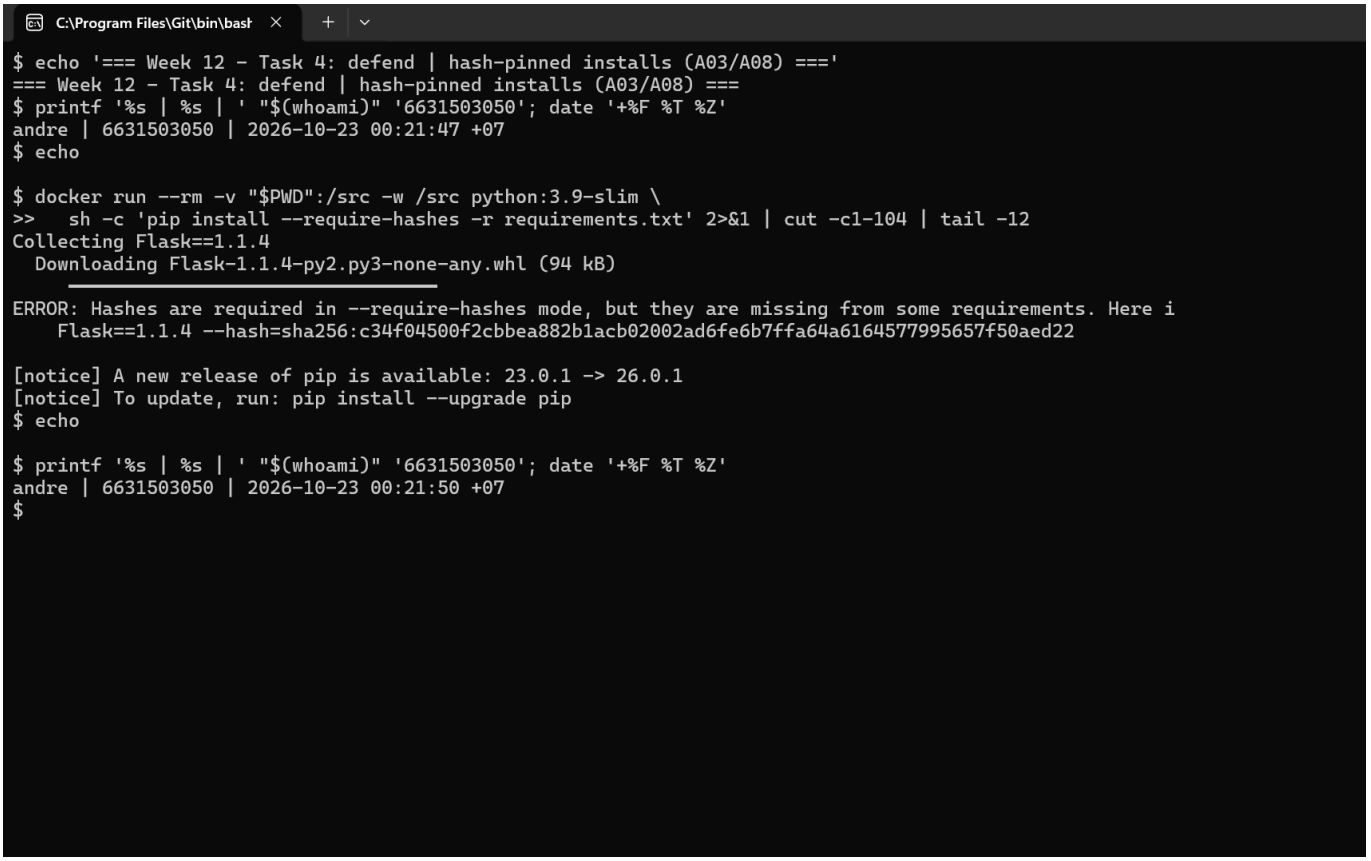
```
has a hash.)
    Flask==1.1.4 --
hash=sha256:c34f04500f2cbbea882b1acb02002ad6fe6b7ffa64a6164577995657f50aed22
```

Why each defense works:

- **Pin + hashes** — once `--require-hashes` (or a committed hash-locked file) is in force, a package whose artifact doesn't match the pinned `sha256` **from either index** can't install silently; a substituted look-alike fails the hash check. (The refusal above is that mechanism engaging — it won't install anything until every requirement is hash-pinned.)
- **Single trusted index** — `--index-url` queries exactly one source, so a same-named package on any other index is never fetched regardless of its version (the Task 2 single-index verdict).
- **Namespace scoping** — reserving/namespacing the internal name (or claiming the public name) means a public look-alike can't collide with it in the first place.
- **Provenance gate** — the `cosign verify` from `sign.sh` runs as a required check before a simulated deploy; an image that isn't signed by the expected identity is rejected, so a tampered artifact never ships.

Most effective defense: a **single trusted (proxied) index plus a hash-pinned lockfile**. Together they close the three cheapest, most common attacks at once — dependency confusion (no second index to shop), typosquatting/substitution (hash mismatch is refused), and silent upgrades (version + hash are fixed) — for very little cost, which is why it's the highest-leverage control for a small team.

Simulation before/after (merged vs single): merged `--extra-index-url` installs the public `99.0.0` (confusion); the single `--index-url` defense installs the private `1.4.0` (the public look-alike is never queried) — same version pair, opposite verdict.



Part 4 — Reflection

1. **Mapping:** table — finding | tool (`sca_scan.sh` / `sign.sh`) | OWASP 2025 id | CWE | fix.

   A:

Finding	Tool	OWASP 2025	CWE	Fix
Outdated, known-vulnerable dependencies (Jinja2 2.11.3, urllib3 1.26.4, Flask 1.1.4, ...)	`sca_scan.sh` (`trivy fs` / `pip-audit`)	**A03**	CWE-1104 / CWE-1395	Bump each to its fixed version; pin + hash-lock; SCA gate in CI
Dependency confusion (public look-alike `acme-internal-utils`)	`dependency-confusion.md` / Task 2 sim	**A03**	CWE-1357 / CWE-829	Single `--index-url`; `--require-hashes`; namespace/reserve the name
Unsigned / tamperable image (no integrity proof)	`sign.sh` (`cosign`)	**A08**	CWE-345 / CWE-494	Keyless sign + `cosign verify` gate before deploy
No component inventory / provenance	`sign.sh` (`trivy` SBOM)	**A08**	CWE-1357	CycloneDX SBOM as a build artifact; provenance attestation

2. **Real breach:** analyze the **XZ Utils backdoor (CVE-2024-3094)**. Which OWASP/CWE applies, and would an SBOM + signing + provenance gate have caught a maliciously-modified upstream dependency?

   A: In the **XZ Utils backdoor (CVE-2024-3094, 2024)**, a maintainer who had spent ~2 years earning commit trust ("Jia Tan") slipped an obfuscated backdoor into the `xz`/`liblzma` **release tarballs** — hidden in binary "test" files and injected by the tarball's own build scripts (not the git source) — that hooked `sshd` (via the `systemd`→`liblzma` link) to allow remote code execution. It's **OWASP 2025 A03** (Software Supply Chain Failures) / **A08** (integrity), with **CWE-506** (embedded malicious code) plus **CWE-1357 / CWE-829**. Would SBOM + signing + provenance have caught it? Mostly **no, as usually deployed**, and that's the important lesson: an **SBOM** lists `xz 5.6.0` — exactly the version you expected, because the malware shipped *inside* a legitimately-named, legitimately-versioned release; **signing** doesn't help when the *trusted maintainer is the attacker* — the release was signed by the real project identity; and provenance would attest it was built by the real `xz` project. Where integrity controls **do** bite: the backdoor lived in the **release tarball but not the git source**, so **SLSA L3 build-from-source + reproducible builds + source-vs-tarball diffing** would have exposed the discrepancy — which is exactly how a Postgres engineer eventually caught it (an anomaly, not a signature check). Compare **SolarWinds** (the *build system* was compromised and signed malicious output — here a hardened SLSA L3 builder genuinely helps) and **Log4Shell** (a *vulnerability*, not malicious insertion — SBOM makes you fast at finding every affected build). Net: signing/SBOM prove *who built what*, but can't by themselves catch a malicious insider at the trust root; defense needs reproducible builds, build-from-source, and reduced single-maintainer trust.

3. **Best mitigation + SLSA self-assessment:** one paragraph.

   A: With this lab's defenses — hash-pinned dependencies pulled through a **single trusted index**, a **CycloneDX SBOM** generated as a build artifact, and `cosign verify` **as a gate before deploy** — I'd self-assess at roughly **SLSA Build L2**: I have signed provenance/signatures and I verify them before shipping, but I'm **not L3** because there's no hardened, isolated, ephemeral builder making the provenance non-forgeable (a compromised build step could still forge it) and no source-track controls. The single highest-leverage control for a small team is the **hash-pinned lockfile served through one curated/proxied index**: in one cheap, low-friction change it closes dependency confusion (no second index to shop), typosquatting and substitution (a hash mismatch is refused), and silent/malicious upgrades (version + hash are fixed) — the attacks that actually hit most projects — whereas full SLSA L3 and reproducible builds are higher-value but much higher-cost next steps.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–3: SCA findings, confusion proof, SBOM/verify)	40
Defense (Task 4: pinning/hashes, single index, scoping, provenance gate)	25
Reflection (Part 4: mapping, XZ breach, SLSA self-assessment)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted. *The stamp runs in the host shell where the lab commands are issued, so `whoami` reads my account; the SCA and signing steps run in throwaway `--rm` containers and appear in the same transcript.*
- **Personalized flag (if this lab issues one):** This lab issues **no flag** — there is no live registry and no exploit payload; the graded evidence is the SCA findings, the resolver-simulation verdicts, and the SBOM/verify transcript. *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*

  1. What did you do, and **why did the vulnerability work**?

     A: I scanned a set of deliberately outdated dependencies and got a worklist of known CVEs, walked the resolver's "highest version wins" rule to see a public look-alike beat the private package, and produced an SBOM + signature to prove what I shipped. The weaknesses work because the toolchain trusts whatever it's told to fetch: SCA only matters because nobody had

bumped the pins, confusion works because the resolver scores packages by name + version with no notion of a trusted source, and an unsigned image can't prove it wasn't swapped.

2. **Why does your fix actually stop it** — and what could still break it?

   A: A single trusted index plus hash-pinned requirements means the resolver never queries a second registry and refuses any artifact whose hash doesn't match, so confusion and substitution both fail; bumping to fixed versions clears the SCA findings; and a `cosign verify` gate rejects any image not signed by the expected identity. What could still break it: the hashes/lockfile must be kept current or they rot; a malicious **insider at the trust root** (the XZ case) ships a signed, correctly-named, correctly-hashed backdoor that these controls won't catch; and a transitive dependency added later re-opens the SCA gap unless CI re-scans every build.

Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

   AI answer (fix for dependency confusion):

   ```
   # Put the private index first so pip prefers it over public PyPI:
   pip install \
     --index-url https://pypi.org/simple \
     --extra-index-url https://pypi.internal.acme/simple \
     acme-internal-utils
   ```

   > "Listing your private index means pip prefers packages from it, so the internal `acme-internal-utils` is chosen over the public one."

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

   A: The reasoning "**Listing your private index means pip prefers packages from it**" is **false**, and the "fix" changes nothing. pip's resolver does **not** honor index order or source priority with `--extra-index-url`: it aggregates candidate versions from **all** configured indexes and installs the **highest version**, regardless of which index it came from. So the attacker's public `acme-internal-utils==99.0.0` still beats the private `==1.4.0` — the command is still in the vulnerable **merged** mode (`--extra-index-url`), which is exactly the setup that *causes* confusion. The answer treats "list it first" as a priority when pip has no such concept; it's a confident, plausible, and wrong fix.

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

   Correct version:

   ```
   # 1) Single trusted index only — never merge public + private:
   pip install \
   ```

```
    --index-url https://pypi.internal.acme/simple \
    --require-hashes \
    -r requirements.lock         # committed, every package hash-pinned
#    (the internal index proxies/curates the public packages you allow)
```

Verified: in the Task 2 simulation's **single-index** mode the public `99.0.0` is never queried, so the private `1.4.0` installs; and `--require-hashes` against a hash-pinned lockfile refuses any artifact whose `sha256` doesn't match, so a substituted package from either source fails to install. **Why the AI was insufficient:** it assumed `--extra-index-url` order confers priority, but pip picks the highest version across all indexes, so its command stays in the exact merged mode that enables confusion. The real fix removes the second index entirely (single `--index-url`) and adds hash-pinning, so there's neither a public source to shop nor a way for a non-matching artifact to install.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

A: The lab builds a tiny app on top of a pile of deliberately out-of-date libraries, and the real weakness is how blindly the package manager fetches those libraries: it picks a package by its name and the highest version number it can find across every registry it's told to check, and it runs the package's install script on your machine before your program even starts. So a stranger who publishes a package with your internal name and a bigger version number (or a name one typo away from a popular one) can get their code to run as you at install time. Signing and bills-of-materials matter because, without them, you can't even tell which exact pieces you shipped or prove nobody swapped one out.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

Final prompt: "Give me the pip setup that makes dependency confusion impossible for an internal package `acme-internal-utils`. Requirements: do **not** merge public and private indexes — use a **single** `--index-url` (a private proxy that curates allowed public packages); require hashes from a committed lockfile so any substituted artifact fails to install; and do **not** rely on keeping the internal version number high (that is not a defense, since the attacker can publish higher). Return the exact pip command(s) and one sentence on why it blocks substitution." **Verified result:** the AI returned a single `--index-url` (private proxy) with `--require-hashes -r requirements.lock` and no `--extra-index-url`. Checked against the Task 2 simulation: in single-index mode the public `99.0.0` is never considered, so `1.4.0` installs; and `--require-hashes` refuses any artifact whose hash isn't pinned — so the substitution the merged-mode exploit relied on can't happen. Naming "not relying on a high version number" in the prompt is what stopped the AI from regressing to the useless "bump the internal version" answer. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*